

# EngineIQ

Engineering Intelligence Platform

## Architecture Addendum

Python Platform · Build Sequence · Deployment & IP Protection

Confidential · Version 1.0 · March 2026

---

## 9. Python Platform — Architecture & Integration

Python does not replace any core EngineIQ component. It operates as the intelligence, automation, and analytics layer that wraps the .NET platform — handling everything the core platform should not own. This section defines the Python services, their boundaries, and how they are sequenced across milestones.

### 9.1 Python Layer Overview

EngineIQ's Python layer is composed of five discrete service groups. Each runs independently, communicates via REST or shared PostgreSQL/Service Bus, and is containerised for deployment alongside the core platform.

| Service Group         | Responsibility                                                                       | Milestone Introduced                  |
|-----------------------|--------------------------------------------------------------------------------------|---------------------------------------|
| Prompt Lab            | Azure OpenAI prompt testing, token cost measurement, diff simulation                 | Milestone 0 — before first PR comment |
| Webhook Simulator     | Fire realistic GitHub PR events at the API endpoint for load and integration testing | Milestone 0 — Day 1 tooling           |
| Reporting Engine      | 30-day metrics email, drift score charts, PDF report generation                      | Milestone 4 — Dashboard & Metrics     |
| Load Testing Service  | 10× load simulation, SLA validation, dead-letter queue stress testing                | Milestone 5 — Production Hardening    |
| Analytics & ML Engine | Benchmark dataset, drift clustering, sprint failure prediction model                 | Phase 5 — Delivery Risk Intelligence  |

### 9.2 Python Service Definitions

#### 9.2.1 Prompt Lab (engineiq-prompt-lab)

A local developer tool — not deployed to clients. Runs as a Jupyter-compatible script environment for iterating on Azure OpenAI prompts outside the main .NET codebase. Keeps prompt changes out of the production build cycle.

| Attribute     | Detail                                                                                |
|---------------|---------------------------------------------------------------------------------------|
| Tech Stack    | Python 3.12, httpx, tiktoken, rich, python-dotenv                                     |
| Key Libraries | tiktoken (token cost), httpx (API calls), rich (terminal output formatting)           |
| Inputs        | Raw PR diff file (.patch), prompt template (.txt), Azure OpenAI credentials from .env |
| Outputs       | Structured finding list, token count, estimated cost in ZAR, acceptance rate score    |
| Deployment    | Local only. Never shipped to clients. Run from /tools/prompt-lab/ in the monorepo.    |

### 9.2.2 Webhook Simulator (engineiq-webhook-sim)

Fires realistic GitHub PR webhook payloads at the EngineIQ API endpoint. Used during development, integration testing, and Milestone 5 production hardening. Removes dependency on real GitHub events during testing cycles.

| Attribute    | Detail                                                                                 |
|--------------|----------------------------------------------------------------------------------------|
| Tech Stack   | Python 3.12, httpx, hmac, hashlib, faker                                               |
| Capabilities | HMAC-SHA256 signed payloads, configurable PR size, multi-tenant simulation, burst mode |
| Outputs      | HTTP response log, latency histogram, SLA pass/fail report per run                     |
| Deployment   | CI/CD pipeline (GitHub Actions) and local dev. Never shipped to clients.               |

### 9.2.3 Reporting Engine (engineiq-reporting)

Generates the 30-day metrics email report, architecture drift score charts, and the board-level PDF report (Phase 5). Pulls data from the PostgreSQL Audit Store and renders output via templated HTML and PDF. Runs as a scheduled Azure Container App on a nightly cron.

| Attribute   | Detail                                                                                         |
|-------------|------------------------------------------------------------------------------------------------|
| Tech Stack  | Python 3.12, pandas, matplotlib, jinja2, weasyprint, psycpg2, sendgrid                         |
| Data Source | PostgreSQL Audit Store. Tenant-isolated queries via TenantId parameter.                        |
| Outputs     | HTML email (SendGrid), drift score chart PNG (embedded), PDF board report (Phase 5)            |
| Schedule    | Nightly metrics aggregation, monthly report dispatch. Triggered via Azure Container Apps Jobs. |
| Deployment  | Containerised. Deployed to Azure Container Apps (SA North). Included in client environment.    |

### 9.2.4 Load Testing Service (engineiq-loadtest)

Used during Milestone 5 production hardening. Simulates 10× expected webhook volume using Locust to validate the 90-second PR review SLA and confirm Service Bus dead-letter queue handling under pressure. Never shipped to clients.

| Attribute        | Detail                                                                                           |
|------------------|--------------------------------------------------------------------------------------------------|
| Tech Stack       | Python 3.12, locust, httpx, azure-servicebus                                                     |
| Test Scenarios   | Burst mode (50 PRs/min), sustained load (10 PRs/min for 60 min), large PR chunking (>2000 lines) |
| Metrics Captured | p50/p95/p99 latency, SLA breach rate, dead-letter queue depth, Azure OpenAI token consumption    |

| Attribute  | Detail                                                                                 |
|------------|----------------------------------------------------------------------------------------|
| Deployment | Run from CI/CD (GitHub Actions) against staging environment only. Not in client image. |

### 9.2.5 Analytics & ML Engine (engineiq-analytics)

Introduced in Phase 5. Processes accumulated cross-tenant finding metadata (never source code) to produce benchmark datasets, industry-level comparisons, and the sprint failure probability model. This is the engine behind EngineIQ's long-term data moat.

| Attribute      | Detail                                                                                         |
|----------------|------------------------------------------------------------------------------------------------|
| Tech Stack     | Python 3.12, pandas, scikit-learn, numpy, sqlalchemy, FastAPI (model serving)                  |
| Phase 5 Output | Sprint failure probability score, DORA metrics calculator, engineering health composite score  |
| Phase 6 Output | SA FinTech/HealthTech benchmark dataset, incident-to-finding correlation, loop-close reporting |
| Model Serving  | Lightweight FastAPI endpoint consumed by EngineIQ.Worker. Not exposed publicly.                |
| Data Scope     | Anonymised, aggregated finding metadata only. Never source code, never PII, POPIA-compliant.   |
| Deployment     | Containerised. Azure Container Apps (SA North). Included from Phase 5 onward.                  |

## 10. Python Build Sequence — When to Write What

Python components are written in-parallel with .NET milestones — not after. Each Python service is introduced at the earliest milestone where it delivers direct value. The rule is simple: write the Python tool before you need it, not after you feel the pain of not having it.

### 10.1 Milestone-by-Milestone Build Schedule

| When                                     | Python Component         | What to Build                                                                                    | Why Now                                                                                                          |
|------------------------------------------|--------------------------|--------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <b>Milestone 0</b><br><b>Weeks 1–2</b>   | Webhook Simulator        | HMAC-signed PR payloads, diff size variants, multi-tenant headers                                | You cannot test your webhook endpoint without real GitHub events unless you can fake them. Build this on Day 1.  |
| <b>Milestone 0</b><br><b>Weeks 1–2</b>   | Prompt Lab               | Diff loader, Azure OpenAI call, token counter, ZAR cost estimate, finding extractor              | Iterate on prompts without touching .NET. Validate < R8/review target before wiring into the engine.             |
| <b>Milestone 1</b><br><b>Weeks 3–5</b>   | Rule Validator Script    | Load YAML standards config, run against sample diffs, output false positive rate                 | Test Standards Engine rule accuracy before integrating. Catch false positives before they erode developer trust. |
| <b>Milestone 4</b><br><b>Weeks 13–16</b> | Reporting Engine v1      | PostgreSQL query layer, pandas aggregation, jinja2 HTML email template, SendGrid dispatch        | The 30-day metrics report is the CTO sales asset. It must ship with the dashboard, not after.                    |
| <b>Milestone 5</b><br><b>Weeks 17–20</b> | Load Testing Service     | Locust scenarios: burst, sustained, large-PR. SLA pass/fail report. Dead-letter queue stress.    | Production hardening milestone requires 10× load validation. Locust is the right tool — not ab, not k6.          |
| <b>Phase 2</b><br><b>Month 5–12</b>      | Reporting Engine v2      | Drift score chart (matplotlib), trend email with blurred upgrade CTA, weekly dashboard data push | Drift score upgrade trigger requires visual reporting. The blurred detail section is the growth lever.           |
| <b>Phase 5</b><br><b>Month 15–24</b>     | Analytics & ML Engine v1 | DORA metrics calculator, sprint failure probability model (scikit-learn), FastAPI model server   | Delivery Risk Intelligence cannot ship without a forecasting model. Python is the only viable stack.             |
| <b>Phase 5</b><br><b>Month 15–24</b>     | Board Report Generator   | weasyprint PDF generation, client branding tokens, auto-scheduled monthly dispatch               | Board-level PDF report is the stickiness mechanism. Must be automated — not manually generated.                  |
| <b>Phase 6</b><br><b>Month 18–30</b>     | Analytics & ML Engine v2 | SA benchmark dataset pipeline, incident-to-finding                                               | The data moat. This is what no global competitor can replicate for the SA market.                                |

| When | Python Component | What to Build                              | Why Now |
|------|------------------|--------------------------------------------|---------|
|      |                  | correlation, loop-close report, clustering |         |

|      |                                                                                                                                                                                        |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RULE | Never start a Python component after you feel the pain of not having it. The Webhook Simulator and Prompt Lab are Day 1 items — they make every subsequent milestone faster and safer. |
|------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 10.2 Python Repository Layout

Python tooling lives in the same monorepo as the .NET platform, under a /python top-level directory. Each service has its own package, Dockerfile, and requirements.txt. They share no runtime dependencies with the .NET services.

| Path                | Responsibility                                                        |
|---------------------|-----------------------------------------------------------------------|
| python/webhook-sim/ | Webhook Simulator. Dev + CI use only. Never deployed to client.       |
| python/prompt-lab/  | Prompt testing toolkit. Local dev use only. Never deployed.           |
| python/reporting/   | Reporting Engine. Containerised. Deployed to all client environments. |
| python/analytics/   | Analytics & ML Engine. Containerised. Deployed from Phase 5 onward.   |
| python/shared/      | Shared models, DB client, Azure Key Vault client, logging config.     |
| python/tests/       | pytest unit and integration tests for all Python services.            |

# 11. Deployment Architecture & IP Protection

EngineIQ ships to clients as container images — not source code. This is the fundamental IP protection mechanism. Clients receive a running platform, not a codebase. This section defines the full deployment model, image delivery pipeline, and code security controls across both .NET and Python services.

## 11.1 Core IP Protection Principle

|           |                                                                                                                                                                                                                                                                                                                     |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PRINCIPLE | No client ever receives source code. All .NET and Python services are compiled and containerised. Clients receive Docker images pulled from a private Azure Container Registry. The registry is EngineIQ-controlled. Client environments authenticate via scoped pull tokens — read-only, time-limited, per-client. |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 11.2 Deployment Models

EngineIQ supports two deployment models depending on client tier. The model determines how images are delivered and where data is processed.

| Model               | Tiers                               | Infrastructure Owner            | Notes                                                                                                                                 |
|---------------------|-------------------------------------|---------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| Cloud-Hosted (SaaS) | Starter, Growth                     | EngineIQ (Azure SA North)       | Client has zero infrastructure responsibility. Multi-tenant. Tenant-isolated via RLS and Service Bus.                                 |
| Private Deploy      | Scale (POPIA private deploy add-on) | Client's own Azure subscription | EngineIQ provides Bicep IaC + Docker Compose. Client pulls images from EngineIQ ACR via scoped token. EngineIQ retains image control. |

## 11.3 Container Image Pipeline

All services — .NET and Python — follow the same image pipeline. Source code never leaves the EngineIQ build environment.

| # | Stage          | Detail                                                                                                                                       |
|---|----------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Build (.NET)   | dotnet publish -c Release. Outputs compiled IL binaries. No source files in output layer.                                                    |
| 2 | Build (Python) | Cython compilation for business-logic modules (.pyx → .so). Non-sensitive scripts remain .py. Multi-stage Dockerfile discards source layer.  |
| 3 | Image Assembly | Multi-stage Dockerfile. Stage 1: build. Stage 2: runtime-only. Source files are never COPY'd to Stage 2. Final image contains binaries only. |

| # | Stage              | Detail                                                                                                                                           |
|---|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| 4 | Image Signing      | All images signed with Notation (Azure Container Registry Notary). Client environments validate signature on pull. Tampered images are rejected. |
| 5 | Registry Push      | Pushed to EnginelQ-owned Azure Container Registry. Repository per service. Tags: {version}-{env}. e.g. 1.4.2-prod.                               |
| 6 | Client Token Issue | Per-client scoped pull token issued via ACR token API. Read-only. Expires in 90 days. Rotated automatically by Tenant Management API.            |
| 7 | Client Pull        | Client environment (Azure Container Apps or private Docker host) pulls images using scoped token. Signature validated. Image runs.               |

## 11.4 Python-Specific IP Protection

Python source is readable by default — unlike compiled .NET IL. Additional protection is applied to business-critical Python modules before containerisation.

| Module Type               | Protection Method                 | Applies To                                                                                        |
|---------------------------|-----------------------------------|---------------------------------------------------------------------------------------------------|
| Business Logic            | Cython compilation (.pyx → .so)   | ML model scoring, benchmark calculation, drift algorithm, report generation logic                 |
| Configuration & Templates | Excluded from final image layer   | Jinja2 templates, YAML defaults — loaded from Azure Blob Storage at runtime, not baked into image |
| Infrastructure Scripts    | Plain .py (acceptable)            | DB connection wrappers, Key Vault client, logging setup — no proprietary logic                    |
| Internal Tooling          | Not containerised — never shipped | Webhook Simulator, Prompt Lab, Load Testing Service — development tools only                      |

### NOTE

Cython compilation is not perfect obfuscation, but it raises the effort bar significantly. Combined with multi-stage Docker builds that discard source layers, it is sufficient protection for commercial SaaS deployment. Enterprise-grade reverse engineering protection (PyArmor) can be added in Phase 3 if competitive pressure warrants it.

## 11.5 Secrets & Credential Management

No credentials are baked into images. All secrets are injected at runtime from Azure Key Vault via managed identity. This applies to both the cloud-hosted and private deploy models.

| Secret                 | Where Stored                 | Access Method                        |
|------------------------|------------------------------|--------------------------------------|
| Azure OpenAI API Key   | EnginelQ Key Vault (per env) | Managed Identity — no env vars       |
| GitHub App Private Key | EnginelQ Key Vault           | Managed Identity. Rotated quarterly. |

| Secret                       | Where Stored                    | Access Method                                               |
|------------------------------|---------------------------------|-------------------------------------------------------------|
| PostgreSQL Connection String | Key Vault (per environment)     | Managed Identity. Private endpoint — no public DB exposure. |
| ACR Pull Token (client)      | Issued by Tenant Management API | Scoped read-only. 90-day expiry. Auto-rotated.              |
| SendGrid API Key (reporting) | Key Vault                       | Managed Identity. Per-environment key.                      |
| Client Webhook Secret (HMAC) | Key Vault, per-tenant entry     | Retrieved by Webhook Listener on startup per tenant.        |

## 11.6 Private Deploy — Client Delivery Package

For Scale tier clients on the private deploy option, EngineIQ ships a deployment package. This package contains everything the client needs to run EngineIQ in their own Azure subscription — but no source code.

| Package Component           | Contents & Notes                                                                                                     |
|-----------------------------|----------------------------------------------------------------------------------------------------------------------|
| docker-compose.yml          | Defines all EngineIQ services. Images referenced by digest (not just tag) to prevent substitution.                   |
| infra/bicep/                | Azure Bicep IaC templates. Provisions Container Apps, Service Bus, PostgreSQL, Redis, Key Vault. One-command deploy. |
| config/standards-templates/ | Default YAML architecture standards configs. Client-editable. Not IP — this is configuration.                        |
| acr-pull-token.json         | Scoped ACR pull credentials. Generated per-client by Tenant Management API. 90-day expiry.                           |
| INSTALL.md                  | Step-by-step deployment guide. Azure CLI commands. Key Vault secret population instructions.                         |
| DPA-template.docx           | Data Processing Agreement template. POPIA-compliant. Pre-filled with EngineIQ data handling terms.                   |

|                 |                                                                                                                                                                                                                                                               |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SECURITY</b> | Images in docker-compose.yml are pinned by digest (sha256:...), not by tag. This prevents a client from substituting a different image version without EngineIQ's knowledge. The signed image validation step in the container runtime catches any tampering. |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 11.7 Licence Enforcement

Licence enforcement is lightweight but meaningful. It does not require complex DRM — the delivery model itself is the primary control. The following additional controls are applied.

| Control               | How It Works                                                                                                                                                                                                               |
|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ACR Token Expiry      | Pull tokens expire every 90 days. Non-paying clients cannot pull updated images. Platform degrades to last pulled version — a natural forcing function for renewal.                                                        |
| Tenant Status Check   | Every EngineIQ.Worker startup checks Tenant billing status via Tenant Management API. Suspended tenants are blocked from processing new jobs. No hard shutdown — pending jobs complete, new ones queue but do not process. |
| Feature Flags by Plan | Architecture Linter (Growth+), Delivery Forecasting (Scale) are feature-flagged server-side. Not in config the client can edit — enforced by Tenant Management API response at runtime.                                    |
| Telemetry Heartbeat   | Private deploy clients emit a lightweight heartbeat (PR count, job count — no code data) to EngineIQ's monitoring endpoint. Used to detect unauthorised instances. POPIA-disclosed in DPA.                                 |

## 11.8 Updated System Layers — Including Python

The following table extends Section 2.2 of the Architecture Document to include the Python layer as an explicit platform tier.

| Layer | Name                   | Responsibility                                                                      | Technology                                                                                       |
|-------|------------------------|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| 1     | Ingestion              | Receive GitHub webhooks, validate HMAC, enqueue PR jobs                             | .NET 8 Web API, Azure Service Bus                                                                |
| 2     | Context                | Index repo structure, detect architecture style, build semantic context             | Azure AI Search, Redis Cache                                                                     |
| 3     | Review Engine          | Deterministic rules + AI analysis, severity scoring                                 | Azure OpenAI, .NET Background Worker                                                             |
| 4     | Feedback               | Classify findings, format PR comments, post to GitHub                               | GitHub REST API, .NET 8                                                                          |
| 5     | Intelligence           | Store findings, serve dashboard, generate trend reports                             | PostgreSQL, .NET 8 API, React                                                                    |
| 6     | <b>Python Platform</b> | Reporting, analytics, ML forecasting, benchmark dataset, board-level PDF generation | Python 3.12, pandas, scikit-learn, FastAPI, jinja2, weasyprint — Azure Container Apps (SA North) |